GLA PMS – AI-Assisted Multi-Portal Project Management System

A Project Synopsis submitted in partial fulfilment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science and Engineering

Specialization in

Artificial Intelligence and Machine Learning

By

Aryan Singh 2415500105

Samarth Verma 2415500410

Under the Guidance of

Mr. Preshit Mangesh Desai

Ms. Anushka Shukla

Department of Computer Engineering & Applications

Institute of Engineering & Technology



GLA UNIVERSITY
MATHURA

Accredited with A+ Grade by NAAC

12-B Status from UGC

GLA University

Mathura- 281406, INDIA

October, 2026

DECLARATION

We hereby declare that the work which is being presented in the B.Tech. Project **"GLA PMS – AI-Based Multi-Portal Project Management System"**, in partial fulfilment of the requirements for the award of the **Bachelor of Technology** in Computer Science and Engineering Specialization in Artificial Intelligence and Machine Learning and submitted to the Department of Computer Engineering and Applications of GLA University, Mathura, is an authentic record of our own work carried out under the supervision of Mr. Preshit Mangesh Desai & Ms. Anushka Shukla.

The contents of this project report, in full or in parts, have not been submitted to any other Institute or University for the award of any degree.

Sign: _____
Name of Candidate: Aryan Singh
Univ Roll No.: 2415500105
Course: B.Tech CS(AIML)

Sign: _____
Name of Candidate: Samarth Verma
Univ Roll No.: 2415500410
Course: B.Tech CS(AIML)

CERTIFICATE

This is to certify that the above statements made by the candidate are correct to the best of my/our knowledge and belief.

Mentor	**Mentor**
Mr. Preshit Mangesh Desai	**Ms. Anushka Shukla**
Dept. of Computer Engg. & App.	Dept. of Computer Engg. & App.

Project Co-ordinator
Mr. Preshit Mangesh Desai
Associate Professor
Dept. of Computer Engg. & App.

Date: _____

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to everyone who supported and guided us throughout the development of this project. First and foremost, we extend our heartfelt thanks to our respected guide, Mr. Preshit Mangesh Desai & Ms. Anushka Shukla, for continuous encouragement, valuable feedback and insightful suggestions at every stage of the project titled **"GLA PMS – AI-Assisted Multi-Portal Project Management System"**. The guidance received has been fundamental in shaping both the direction and the outcome of this work.

We are deeply thankful to the **Department of Computer Engineering and Applications** and the AIML project coordination office for trusting us with a real academic workflow, for sharing semester rosters, problem-statement catalogues and evaluation rubrics, and for the candid feedback that turned an idea into a system used by students, mentors and evaluators every day.

We also thank the **Head of the Department, Prof. Ashok Bhansali**, for his constant support and motivation, the faculty mentors and panel members who piloted the portals and reported issues, and our classmates whose questions and bug reports made the platform better with every release. Lastly, we owe our deepest appreciation to our families for their patience and encouragement throughout this journey.

Aryan Singh
2415500105

Samarth Verma
2415500410

ABSTRACT

GLA PMS is an AI-assisted, multi-portal project management system that digitises the complete academic project lifecycle of the Department of Computer Engineering & Applications at GLA University — from roster import and team formation to time-bound evaluation and published results. Before the platform, semester projects were coordinated through spreadsheets, e-mail threads and paper score-sheets: rosters drifted between copies, popular problem statements were claimed by too many teams, mentor matching took a day of manual work, and evaluation marks had to be consolidated by hand.

The system provides four role-specific web portals — **Admin (Coordinator), Student, Mentor and Panel** — and a public landing site, built with React 19 and Vite, backed by **four independent FastAPI micro-services** that share one semester-scoped **Supabase PostgreSQL** schema. Separate services isolate the four user populations by trust level, load pattern and authentication model, while the shared schema keeps every team, mark and message consistent. Coordinators import rosters and configure semesters, presentation cycles and rubrics; students form teams, reserve problem statements and submit proposals and synopses; mentors supervise teams with repository analytics; and panel members score teams inside a server-enforced evaluation window that issues tamper-evident receipts.

An **intelligence and automation layer** reduces coordinator effort: an explainable, rank-aware **mentor allocation engine** built on an 11-cluster domain taxonomy; **LLM-based problem-statement ingestion** that parses Word, PDF and Excel catalogues and classifies them using GPT-4o-mini; a **GitHub analytics agent** that reports code quality, security, originality and per-student contribution; **fuzzy roster matching** with Jaro-Winkler and Damerau-Levenshtein similarity; a segment-based **communication agent**; and an **escalation agent** that e-mails late evaluators at +5 and +10 minutes and places a Twilio voice call at +15 minutes. Data integrity is enforced inside the database through an atomic, row-locking PostgreSQL function that caps every problem statement at two teams, a canonical team identifier, and freeze, lock and audit controls.

The platform runs as Docker containers on AWS EC2 behind a Cloudflare Tunnel, is delivered through a GitHub Actions pipeline, and is verified by **169 automated test cases** across 26 suites. A structured security review in September 2026 left row-level security deny-by-default, audience-scoped tokens and server-side secrets. GLA PMS shows how pragmatic AI, sound data modelling and defence-in-depth can turn a fragmented academic workflow into a transparent, auditable and scalable system.

TABLE OF CONTENTS

Declaration ii
Certificate iii
Acknowledgement iv
Abstract v
Table of Contents vi
List of Tables viii
List of Figures ix

CHAPTER 1: INTRODUCTION 1
1.1 Overview and Motivation 1
1.2 Objectives 3
1.3 Summary of Similar Applications 4
1.4 Organization of the Project Report 4

CHAPTER 2: SOFTWARE REQUIREMENT ANALYSIS 6
2.1 Introduction to Requirement Analysis 6
2.2 Feasibility Study 6
2.3 Functional Requirements 7
2.4 Non-Functional Requirements 8
2.5 Use Case Descriptions 9
2.6 Module Descriptions 10

CHAPTER 3: SOFTWARE DESIGN 11
3.1 System Architecture Overview 11
3.2 Data Flow Diagrams 12
3.3 UML Diagrams 13
3.4 Database Design 15

CHAPTER 4: IMPLEMENTATION AND USER INTERFACE 19
4.1 Technology Stack Used 19
4.2 Intelligence and Automation Architecture 20
4.3 Core Algorithms and Integrity Mechanisms 23
4.4 Frontend Implementation 24
4.4.1 Admin (Coordinator) Portal 24
4.4.2 Student Portal 25
4.4.3 Mentor Portal 25
4.4.4 Panel Portal 26
4.5 Deployment and DevOps 26

CHAPTER 5: SOFTWARE TESTING 28
5.1 Testing Strategy 28
5.2 Automated Unit and Contract Tests 28
5.3 Black Box Test Cases 29
5.4 White Box Test Cases 29
5.5 Security and Concurrency Testing 30

CHAPTER 6: CONCLUSION 32
6.1 Current Status and Future Work 32

CHAPTER 7: SUMMARY 33
7.1 Summary 33

REFERENCES 34

APPENDIX A: Abbreviations 35

APPENDIX B: Sample API Endpoints 36

LIST OF TABLES

Table 1.1 Comparison of Existing Project Management Approaches
Table 2.1 Feasibility Study Summary
Table 2.2 Functional Requirements of GLA PMS
Table 2.3 Non-Functional Requirements of GLA PMS
Table 2.4 Use Case – Coordinator Imports Roster
Table 2.5 Use Case – Team Leader Reserves a Problem Statement
Table 2.6 Use Case – Panel Member Conducts a Time-Bound Evaluation
Table 3.1 Database Table – semester_config
Table 3.2 Database Table – students
Table 3.3 Database Table – teams_validation (Teams)
Table 3.4 Database Table – mentors
Table 3.5 Database Table – presentation_cycles and presentation_rubrics
Table 3.6 Database Table – evaluations
Table 3.7 Database Table – results
Table 4.1 Technology Stack Summary
Table 4.2 Intelligence and Automation Layer – Agent Responsibilities
Table 4.3 Mentor Allocation Scoring Weights
Table 5.1 Automated Test Suites
Table 5.2 Black Box Test Cases – Portal Workflows
Table 5.3 White Box Test Cases – Evaluation and Allocation Logic
Table 5.4 Security Testing Results

LIST OF FIGURES

Figure 1.1 System Architecture of GLA PMS
Figure 3.1 Level 0 DFD – Context Diagram
Figure 3.2 Sequence Diagram – Time-Bound Panel Evaluation
Figure 3.3 Entity Relationship Diagram (Core Entities)
Figure 4.1 Admin Portal – (a) Sign-in and (b) Dashboard
Figure 4.2 Student Portal – (a) Sign-in with Google SSO and (b) Dashboard
Figure 4.3 Mentor Portal – (a) Profile Creation and (b) Dashboard
Figure 4.4 Panel Portal – Evaluator Sign-in

CHAPTER 1
INTRODUCTION

1.1 Overview and Motivation

Every semester the Department runs project-based courses for large B.Tech cohorts. Each project passes through a long chain of stages — roster upload, team formation, topic selection or a custom proposal, mentor allocation, synopsis milestones, several graded presentation rounds and, finally, result compilation. Every stage has a different owner (coordinator, student, mentor or panel member) and a hard deadline, and in the current academic year two cohorts run concurrently: the 2nd-year batch with teams numbered T1–T125 and the 3rd-year batch with teams T1–T197, choosing from a catalogue of 148 problem statements (50 hardware and 98 software).

Run on spreadsheets and e-mail, this lifecycle suffers from several structural problems:

- **Fragmented records:** rosters, team lists and marks live in separate sheets that drift apart; a name corrected in one copy stays wrong in the others.
- **Topic contention:** popular problem statements get claimed by more teams than allowed, and conflicts surface only after manual cross-checking.
- **Manual mentor matching:** pairing dozens of teams with faculty by domain and load is a day of work that still produces uneven allocations.
- **Evaluation overhead:** panel marks are written on paper or sheets, re-keyed, and totalled by hand; late or missed evaluations are noticed only afterwards.
- **Communication gaps:** announcements, reminders and per-team mails go out from personal inboxes with no record of who received what.
- **No accountability trail:** overrides, freezes and corrections leave no audit trail, so disputes about marks cannot be settled objectively.
- **Concurrent cohorts:** two batches with overlapping team numbers make manual identification of a team error-prone.

GLA PMS was built to solve these problems end to end. It replaces the spreadsheets with one authoritative, semester-scoped database, gives each stakeholder a purpose-built portal, and adds assistive AI where manual effort was highest.

Why This Project?

- **Single source of truth:** a master student roster keyed by university roll number gates sign-up, team formation and every later stage.
- **Role-specific portals:** coordinators, students, mentors and panel members each get only the screens and data their role needs.
- **Assistive, explainable AI:** problem-statement ingestion, domain classification, mentor recommendation and repository analytics are automated, while every final decision stays with a human.
- **Integrity by construction:** capacity limits, evaluation windows, locks and receipts are enforced by the server and the database, not by convention.
- **Transparency:** every e-mail, override and evaluation is logged and traceable.
- **Proven in production:** the platform is deployed at glapms.in and serves the live 2026–27 cohorts.

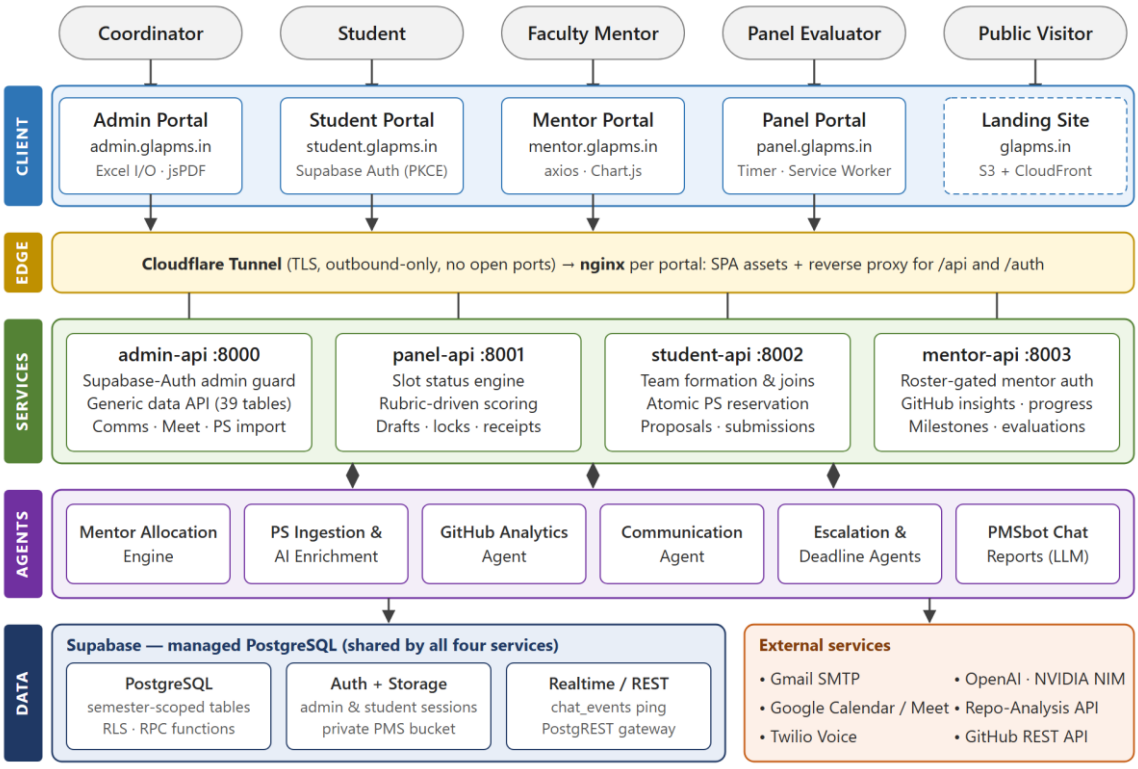


Figure 1.1: System Architecture of GLA PMS

1.2 Objectives

Completed Objectives

1. To build four role-specific portals (Admin, Student, Mentor and Panel) and a public landing site on a shared design language, so that every stakeholder works on one platform.
2. To maintain one master roster per semester, imported from Excel with fuzzy duplicate detection, and to gate sign-up so that only listed students and mentors can register.
3. To implement team formation with join requests and leader roles, and problem-statement reservation with a hard cap of two teams per statement enforced atomically in the database.
4. To automate mentor allocation with an explainable, rank-aware scoring engine that the coordinator can override and lock.
5. To support dynamic presentation cycles with configurable rubrics and weightage, slot scheduling with Google Meet links, and time-bound panel evaluation with drafts, locks and receipts.
6. To compute results automatically from locked evaluations and publish them to students.
7. To provide AI assistance for problem-statement ingestion, classification and enrichment, and repository analytics for mentors and coordinators.
8. To provide segment-based mass communication, per-team mail merge, deadline reminders and escalation of delayed evaluations.
9. To secure the platform with role-scoped authentication, row-level security, data-freeze controls and audit logs, and to deploy it through containerised CI/CD.

Pending Objectives (Future Scope)

- Retire the legacy team tables still read by the mentor and panel services and move every service onto the canonical team registry (the phase-6 migration is prepared and on hold).
- Enable PMS-Chat, the role-aware messaging module, in production; it currently ships behind a build-time flag and per-portal switches.
- Extend automated tests to the admin, mentor and panel APIs and add end-to-end browser tests.

- Introduce a shared cache and uniform rate limiting across all four services.
- Complete the remaining hardening items from the September 2026 audit, such as secret rotation and invite-only sign-up.

1.3 Summary of Similar Applications

Several general-purpose tools are used by universities to manage student projects, but none covers the full lifecycle from roster to result with allocation logic and time-bound panel evaluation. Table 1.1 compares the most common options with GLA PMS.

Table 1.1: Comparison of Existing Project Management Approaches

Platform	Key Features	Team & Topic Allocation	Evaluation Support	Limitations
Spreadsheets + E-mail	Familiar, flexible, no setup	Manual, first-come by e-mail	Paper or sheet scoring, manual totals	Version drift, no integrity checks, no audit trail
Google Classroom	Assignments, announcements, submissions	None; groups created manually	Rubric grading of individual assignments	No team formation, mentors or panel scheduling
Moodle LMS	Courses, group mode, rubric plugins	Group self-selection plugin only	Teacher-side rubric grading	No topic capacity, no panels or slots, generic UX
MS Teams for Education	Chat, meetings, assignments	Manual	Assignment grading	Not designed for a multi-stage project lifecycle
Jira / Trello	Task boards, sprints, workflows	None	None	Built for software teams, not academic assessment
GitHub Classroom	Repository provisioning, autograding	Team repositories	Automated tests only	No proposals, mentors, panels or results
GLA PMS (Ours)	Four role portals, AI assistance, full lifecycle	Atomic reservation (≤ 2 teams per PS), ranked mentor engine	Time-bound panel and mentor evaluation, automatic results	Newly deployed; enhancements ongoing

1.4 Organization of the Project Report

- **Chapter 1 – Introduction:** motivation, objectives, comparison with existing systems and the organisation of the report.
- **Chapter 2 – Software Requirement Analysis:** feasibility study, functional and non-functional requirements, use cases and module descriptions.

- **Chapter 3 – Software Design:** architecture, data flow diagrams, UML diagrams, the entity-relationship model and the database schema.
- **Chapter 4 – Implementation and User Interface:** technology stack, the intelligence and automation layer, core algorithms, the four portals and deployment.
- **Chapter 5 – Software Testing:** automated suites, black-box and white-box test cases, security and concurrency testing.
- **Chapter 6 – Conclusion** and **Chapter 7 – Summary**, followed by references and appendices on abbreviations and API endpoints.

CHAPTER 2
SOFTWARE REQUIREMENT ANALYSIS

2.1 Introduction to Requirement Analysis

Requirement analysis for GLA PMS was carried out with the people who run the project lifecycle: the AIML project coordination office, faculty mentors, panel evaluators and student teams. The existing artefacts — roster spreadsheets, the two problem-statement catalogues for 2026–27, mail templates and score-sheets — were studied to understand the data each stage produces and consumes.

The analysis identified four human roles — **Coordinator (Admin), Student, Mentor and Panel Evaluator** — plus a system actor that runs timed jobs such as reminders and escalations. It also showed that most failures in the manual process were integrity failures (double allocation, lost corrections, unrecorded overrides) rather than missing features, so integrity and auditability were treated as first-class requirements alongside functionality.

2.2 Feasibility Study

Table 2.1: Feasibility Study Summary

Feasibility Type	Description	Analysis	Conclusion
Technical	Availability of frameworks, hosting, AI services and team skills	React 19, Vite, FastAPI, Supabase PostgreSQL and Docker are mature and open source. LLM features are optional enhancements with deterministic fallbacks when no API key is configured.	Feasible
Operational	Fit with the existing workflow of coordinators, faculty and students	Portals map one-to-one to existing roles; roster-gated sign-up removes manual account creation; guided walkthroughs reduce training effort.	Highly Feasible
Economic	Development, hosting and running cost	Open-source stack; one EC2 host runs all containers; managed Supabase with daily backups; GPT-4o-mini calls are low-cost and the token cost of every AI suggestion is computed and returned.	Cost-Effective

Feasibility Type	Description	Analysis	Conclusion
Legal and Ethical	Protection of student and faculty personal data	Access restricted to university accounts, bcrypt password hashing, row-level security, private storage bucket and audit logs; no data is shared with third parties beyond the AI prompts needed for classification.	Feasible
Schedule	Delivery within the academic calendar	Iterative releases: atomic reservations (11 Aug 2026), delegate conversations (14 Aug), domain catalogue (21 Aug), chat toggles and delivery logs (24 Aug), security hardening (3–4 Sep).	Feasible

2.3 Functional Requirements

Table 2.2: Functional Requirements of GLA PMS

S.No.	Requirement	Description
1	**Semester Configuration**	Create batches (year, semester type, branch, academic level) with team-size bounds, number of presentations and stage weightages; activate, lock or delete a batch; several cohorts may be active together.
2	**Master Roster Import**	Excel/CSV upload with alias-based header detection, merged-cell fill-down and fuzzy duplicate detection; idempotent upsert keyed by roll number.
3	**Role-Based Authentication**	Supabase Auth for coordinators and students (Google sign-in restricted to @gla.ac.in), service-issued JWTs for mentors and panels, roster-gated sign-up and 15-minute password-reset links.
4	**Team Formation**	Create a team with the lowest free team number, send and accept join requests, add or remove members, disband; one team per student per semester.
5	**Problem Statement Catalogue**	Import catalogues from DOCX, PDF, Excel or CSV; AI classification into hardware/software, difficulty and domain; filtering by B.Tech year.
6	**Problem Statement Reservation**	The team leader reserves a statement; at most two teams may hold one statement, enforced by an atomic database function.
7	**Custom Proposals**	Leader-only submission with a document link before the deadline; approve, reject or request revision with automatic e-mails; resubmission after rejection.
8	**Mentor Allocation**	Roster-gated mentor sign-up with ranked domains; ranked recommendations with capacity limits; manual override, locking and on-demand assignment e-mails.
9	**Presentation Cycles**	Proposal, mid-term and final cycles with weightage, deadline, status (draft, active, closed, frozen), rubric criteria, optional synopsis upload and mentor marks.
10	**Scheduling**	Slots with date, time, room and panel; Google Meet events for online slots; slot extension that shifts later slots of the same panel or room.

S.No.	Requirement	Description
11	**Time-Bound Evaluation**	Scoring allowed from two hours before a slot until it ends; auto-saved drafts; per-criterion range checks; mandatory remarks; locked submission with a receipt.
12	**Results**	Weightage-scaled aggregation of every cycle into proposal, mid, final and total marks; PDF reports; publication to students.
13	**Communication**	Ten audience segments, nine templates, per-team mail merge with attachments and a per-recipient delivery log.
14	**Notifications and Reminders**	Deadline reminders at three days, one day and the due day; escalation of late evaluations; in-app notifications.
15	**Repository Analytics**	Quality, security, originality, architecture and documentation scores, contributor attribution, duplicate-commit and inactivity reports.
16	**Governance**	System-wide data freeze, team and marks locks, audit logs, absentee handling with make-up slots.

2.4 Non-Functional Requirements

Table 2.3: Non-Functional Requirements of GLA PMS

S.No.	Requirement	Description
1	**Performance**	Hot reads are served from short-lived caches (identity 30 s, active semesters 60 s, panel status 15 s, client table cache 3 s); bulk e-mail is sent in chunks of 25 per SMTP connection; uploads are parsed in memory.
2	**Scalability**	Each service can be scaled and restarted independently; semester scoping lets several cohorts of about 500 students and 200 teams run at the same time.
3	**Security**	HS256 JWTs with a separate audience and secret per service, 15-minute access tokens with rotated refresh tokens and a revocation list, bcrypt hashing, deny-by-default row-level security, CORS allowlists and rate limits on costly AI and analysis endpoints.
4	**Availability**	Containers restart automatically; startup probes detect a stale database key; managed daily backups with seven-day retention and a documented restore runbook.
5	**Data Integrity**	Row-locking reservation function, unique receipt identifiers, one evaluation per panel, team and semester, canonical team_uid foreign keys and idempotent upserts.
6	**Usability**	Responsive layouts, light and dark themes, first-login walkthroughs and precise validation messages on every form.
7	**Maintainability**	Modular routers, pure-logic modules covered by unit tests, numbered SQL migrations that document each fix, and environment-based configuration.
8	**Auditability**	Governance audit log, override logs, communication history with per-recipient delivery records and immutable evaluation receipts.

2.5 Use Case Descriptions

Use Case 1: Coordinator Imports the Roster and Configures a Semester

Table 2.4: Use Case – Coordinator Imports Roster

Field	Detail
Use Case ID	UC-01
Actor	Project Coordinator (Admin)
Precondition	Coordinator is signed in with a server-assigned admin role; the roster spreadsheet is available.
Main Flow	1. Create or activate the semester with team-size bounds and weightages. 2. Upload the roster (XLSX or CSV). 3. The system maps headers by alias, fills merged team cells and validates e-mails and roll numbers. 4. Fuzzy matching routes near-duplicates to a review list. 5. Approved rows are upserted into the students table keyed by roll number.
Alternate Flow	Exact duplicates are reported; near-duplicates (typo or transliteration) wait for coordinator review.
Postcondition	Listed students and mentors can now sign up with their university e-mail.
Exception	If every active batch is frozen, the write is rejected with “System is frozen. No modifications allowed.”

Use Case 2: Team Leader Reserves a Problem Statement

Table 2.5: Use Case – Team Leader Reserves a Problem Statement

Field	Detail
Use Case ID	UC-02
Actor	Team Leader (Student)
Precondition	Student is pre-registered, signed in and leader of a team in an active semester.
Main Flow	1. Leader opens “Browse & Reserve Topics”. 2. The system lists year-filtered statements with team count and “1 spot left” or “full” badges. 3. Leader selects a statement. 4. student-api calls reserve_problem_statement_atomic with capacity 2. 5. The database locks the statement and team rows, counts holders and links the team. 6. Title, description and domain are copied to the team.
Alternate Flow	If two teams already hold the statement, the request fails with “problem statement is at capacity”.
Postcondition	teams_validation.problem_statement_id points to the reserved statement.
Exception	A non-leader receives 403 “Only the team leader can reserve problem statements”.

Use Case 3: Panel Member Conducts a Time-Bound Evaluation

Table 2.6: Use Case – Panel Member Conducts a Time-Bound Evaluation

Field	Detail
Use Case ID	UC-03
Actor	Panel Evaluator

Field	Detail
Precondition	Panel account is active and the team has a slot in an active cycle; current time lies between two hours before the slot and its end.
Main Flow	1. Evaluator opens the team from "My Slots". 2. The session returns the rubric and a 30-minute timer. 3. Scores are auto-saved as drafts. 4. Evaluator submits scores with remarks. 5. The server checks each criterion (0 to maximum) and the remarks. 6. The evaluation is stored locked with receipt REC-xxxxxxxxxx, results are recomputed and the draft is deleted.
Alternate Flow	The evaluator extends a running slot by 5 or 10 minutes; later slots on the same panel or room shift automatically.
Postcondition	The slot shows "Submitted" and the team's results are updated.
Exception	Outside the window: 403 time-bound control. Already submitted: "Evaluation already locked for this team". Marks locked: "Marks are locked".

2.6 Module Descriptions

Module 1: Coordinator (Admin) Module

The control plane of the platform. The coordinator configures semesters, imports rosters and problem statements, validates teams, allocates topics and mentors, defines presentation cycles and rubrics, schedules slots with Meet links, monitors live evaluations, publishes results, sends communications and controls freezes and audit logs.

Module 2: Student Module

Covers the student journey from sign-up to results: team formation and join requests, topic reservation or a custom proposal, synopsis milestones, presentation submissions, repository analysis, schedules, marks, feedback, notifications and extra-activity records.

Module 3: Mentor Module

Lets faculty register with ranked domains, view assigned teams and their project blueprints, track repository activity and progress, assign and verify synopsis milestones, and evaluate presentation cycles within rubric bounds.

Module 4: Panel Evaluation Module

Gives evaluators a narrow, time-critical interface: the day's slot queue, the rubric for each team, the timed evaluation form with drafts, extensions, missed-slot rescheduling and a downloadable submission receipt.

Module 5: Intelligence and Automation Module

Hosts the assistive agents — mentor allocation, problem-statement ingestion and enrichment, repository analytics, roster validation, communication, deadline reminders, evaluation escalation and the PMSbot report generator.

CHAPTER 3
SOFTWARE DESIGN

3.1 System Architecture Overview

GLA PMS follows a layered, service-oriented architecture (Figure 1.1). Five browser applications sit on top of four FastAPI services, which share one PostgreSQL schema hosted on Supabase:

- **Presentation Layer:** five React 19 single-page applications built with Vite 8 and Tailwind CSS 4 — the Admin, Student, Mentor and Panel portals and a public landing site. Each portal is served by its own nginx container, which also reverse-proxies /api and /auth to its backend, so browsers make same-origin calls.
- **Edge Layer:** a Cloudflare Tunnel terminates TLS at Cloudflare and connects outbound to the EC2 host; no inbound ports are opened and every container port is bound to 127.0.0.1.
- **Application Layer:** admin-api (port 8000), panel-api (8001), student-api (8002) and mentor-api (8003). The split limits blast radius (a flaw in the student service cannot reach coordinator code), matches different load patterns (the panel service is idle for weeks and then serves every evaluator on presentation day) and separates two authentication models.
- **Intelligence and Automation Layer:** agents and background schedulers that run inside the services — allocation, ingestion, analytics, communication, reminders, escalation and chat reports.
- **Data Layer:** PostgreSQL with semester-scoped tables, row-level security and server-side functions, plus Supabase Auth, a private Storage bucket and a Realtime ping table for chat. Services connect with the service-role key, which never reaches a browser.
- **Integration Layer:** Gmail SMTP, Google Calendar API (Meet), Twilio Voice, OpenAI and NVIDIA NIM models, an external Repository Analysis API and the GitHub REST API.

Requests prove identity in one of two ways. Coordinator and student sessions are issued by Supabase Auth and validated by the services against the Supabase user endpoint; the admin

role is read only from server-controlled app_metadata. Mentor and panel tokens are minted by their own services with python-jose (HS256), each carrying its own audience claim and secret, so a token from one service is rejected by the others.

3.2 Data Flow Diagrams

3.2.1 Level 0 DFD – Context Diagram

The context diagram (Figure 3.1) shows GLA PMS as a single process exchanging data with four human entities — coordinator, student, mentor and panel evaluator — and three external systems: AI services, messaging and calendar services, and the GitHub analysis service.

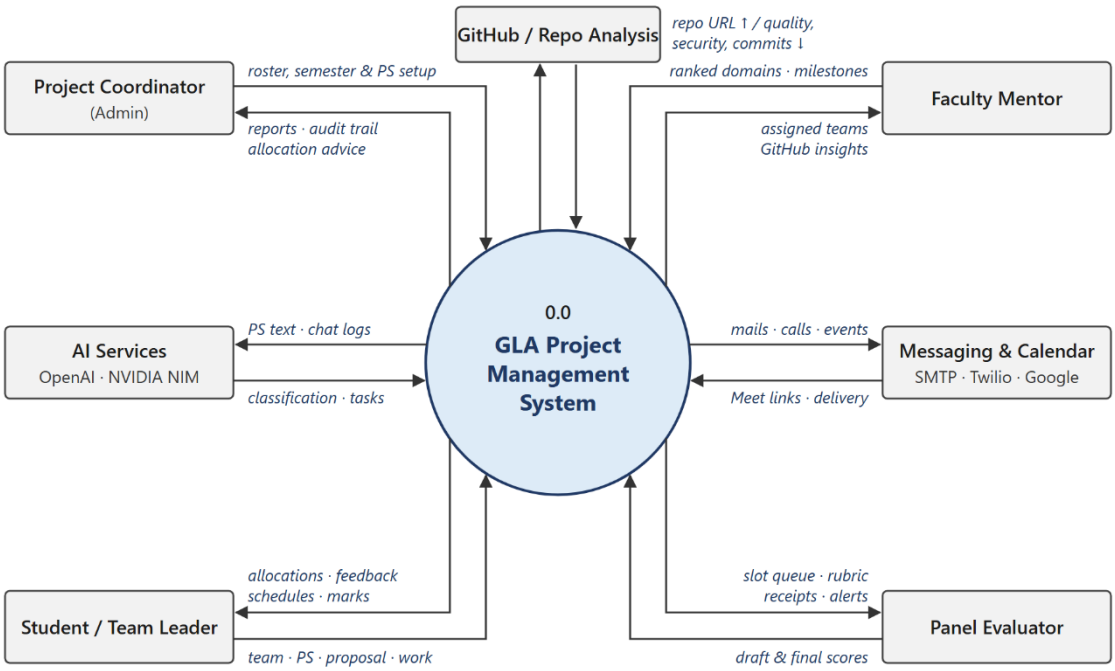


Figure 3.1: Level 0 DFD – Context Diagram

3.2.2 Level 1 DFD – Main Processes

At level 1 the system decomposes into eight processes that follow the lifecycle: (1.0) roster and semester setup, (2.0) team formation, (3.0) problem-statement reservation and proposals, (4.0) mentor allocation, (5.0) presentation scheduling, (6.0) panel and mentor evaluation, (7.0) results computation and publishing, and (8.0) communication and notification. Their principal data stores are students and semester_config (D1), teams_validation and team_memberships (D2), problem_statements and proposals (D3), mentors and domains (D4), schedule_slots and slots (D5), evaluations and evaluation_drafts (D6), results (D7) and communication_history with email_deliveries

(D8). Each process reads the output store of the previous stage, which is why a correction made once in the roster propagates to every later stage.

3.3 UML Diagrams

3.3.1 Use Case Diagram

The use case model contains four actors. The **Coordinator** imports rosters, manages the problem-statement catalogue, allocates mentors, schedules slots, publishes results and sends communications. The **Student** signs in, forms a team, reserves a statement or submits a proposal, submits presentation work and views schedules and marks. The **Mentor** ranks domains at sign-up, monitors teams and GitHub insights, assigns synopsis milestones and evaluates presentations. The **Panel Evaluator** views the slot queue and rubric, performs the time-bound evaluation, saves drafts, locks the submission with a receipt and requests a reschedule for absent teams. A system timer actor triggers deadline reminders and escalations.

3.3.2 Class Diagram

The core classes are SemesterConfig, Student, Team (teams_validation), TeamMembership, ProblemStatement, Proposal, Mentor, Panel, PresentationCycle, Rubric, ScheduleSlot, Evaluation, PresentationEvaluation, Result, Conversation and Message. A SemesterConfig owns many Students, Teams and PresentationCycles; a Team aggregates Students through TeamMemberships (one LEADER and several MEMBERs), holds at most one ProblemStatement and many Proposals, is supervised by a Mentor, and receives many ScheduleSlots and Evaluations; a PresentationCycle defines many Rubric criteria and collects PresentationEvaluations whose per-criterion scores reference those rubrics; and each Team has exactly one Result.

3.3.3 Sequence Diagram – Time-Bound Evaluation

Figure 3.2 traces the most time-critical interaction: a panel evaluator scoring a team. The portal obtains the session and timer, autosaves drafts while the evaluator scores, and on submission the API validates the window, ranges and remarks before writing a locked evaluation, recomputing results and returning a receipt.

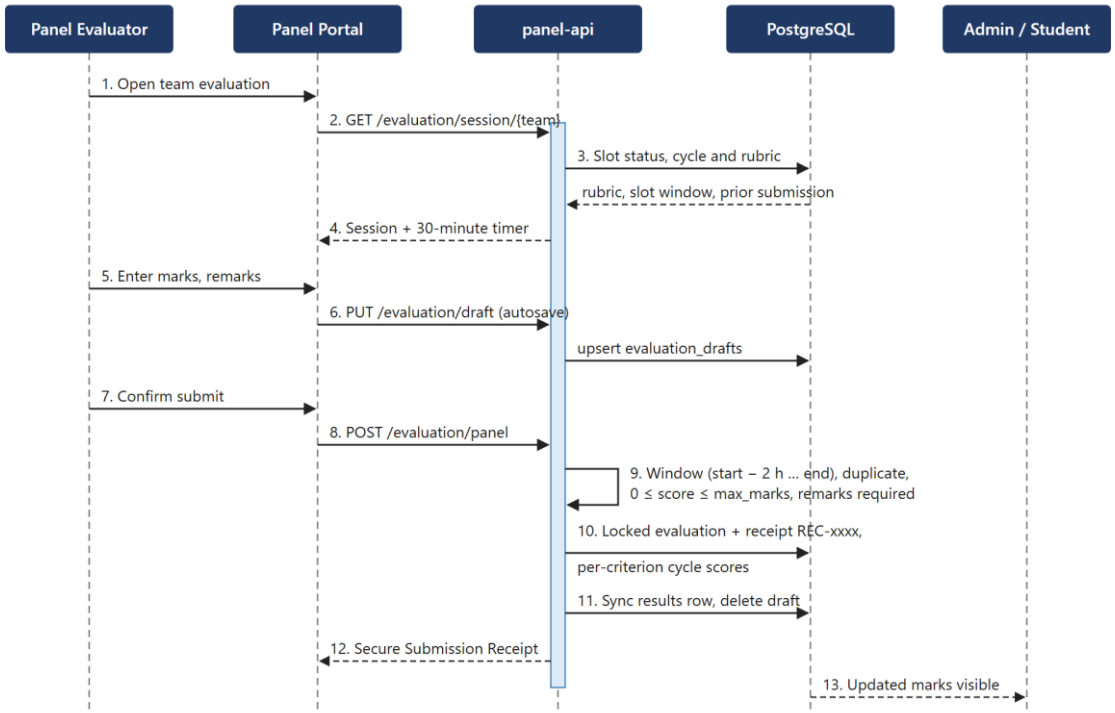


Figure 3.2: Sequence Diagram – Time-Bound Panel Evaluation

3.3.4 Entity Relationship Diagram

Figure 3.3 shows the core entities. teams_validation is the hub: it links students through team_memberships, holds an optional problem statement (each statement is held by at most two teams), receives proposals, schedule slots and evaluations, and owns exactly one results row. semester_config scopes every cohort table, and presentation cycles define the rubrics against which evaluations are recorded.

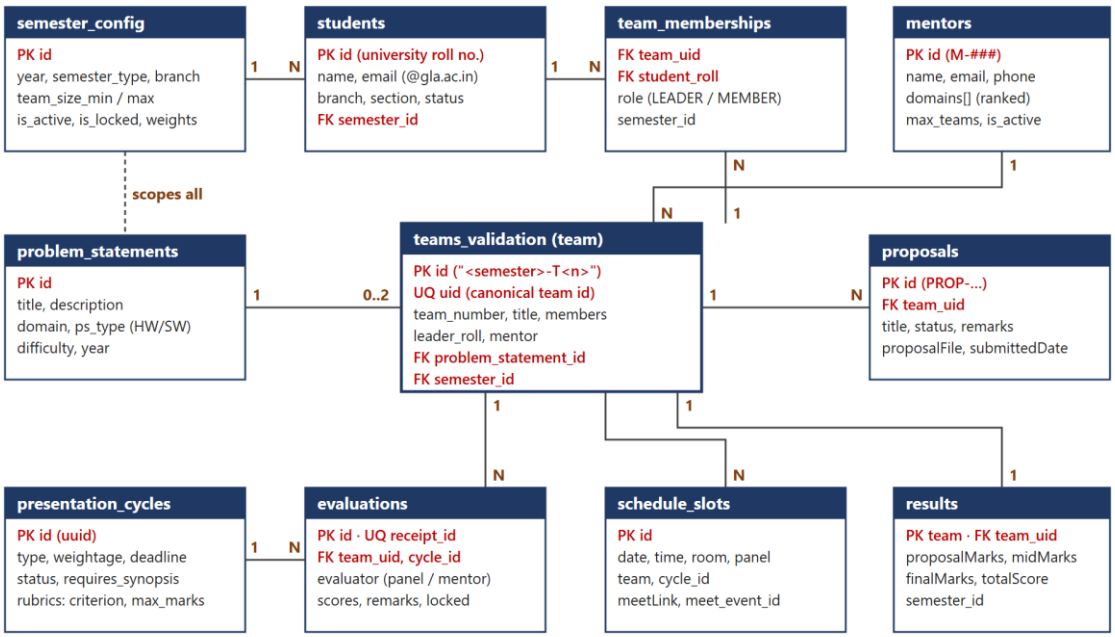


Figure 3.3: Entity Relationship Diagram (Core Entities)

3.4 Database Design

GLA PMS uses PostgreSQL on Supabase as its single source of truth. The repository's SQL defines 49 tables (the production schema audited in September 2026 contains 71 public tables), evolved through numbered migrations such as 003_presentation_cycles, the uid_canonical series and timestamped Supabase migrations. Almost every cohort table carries a semester_id, which lets several batches share tables without mixing. Teams have three identifiers: a text slug "<semester_id>-T<n>", a display team_number and a canonical uid; every team-scoped table now references the uid, which removed cross-cohort mix-ups caused by colliding team numbers. The core tables are described below.

Table 3.1: Database Table – semester_config

Field Name	Data Type	Key / Constraint	Description
id	text	Primary Key	Unique batch identifier (UUID string)
year, semester_type	text	NOT NULL	Academic year and odd/even semester
academic_level, branch	integer, text	—	B.Tech year and branch of the cohort
team_size_min / max	integer	—	Allowed team size bounds
number_of_presentations	integer	—	Graded presentation rounds
proposal / mid / final_weight	integer	—	Weightage of each evaluation stage
is_active, is_locked	boolean	DEFAULT false	Batch activation and system-wide freeze
results_published	boolean	DEFAULT false	Releases results to students
evaluation_parameters	jsonb	DEFAULT '[]'	Default rubric (criterion, max_marks)
pms_chat_student … admin	boolean	NOT NULL DEFAULT false	Per-portal PMS-Chat switches

Table 3.2: Database Table – students

Field Name	Data Type	Key / Constraint	Description
id	text	Primary Key	University roll number (master key)
name	text	NOT NULL	Full name of the student
email	text	Unique	University e-mail used by the sign-up gate
year	integer	—	B.Tech year; filters the problem-statement catalogue
branch, section	text	—	Academic branch and section
status	text	DEFAULT 'Active'	Active, inactive or graduated

Field Name	Data Type	Key / Constraint	Description
semester_id	text	FK → semester_config	Cohort to which the student belongs
phone, bio, skills	text	—	Optional profile attributes

Table 3.3: Database Table – teams_validation (Teams)

Field Name	Data Type	Key / Constraint	Description
id	text	Primary Key	Slug "<semester_id>-T<n>"
uid	uuid	Unique (canonical)	Global team identity referenced by team data
team_number	integer	—	Display number; lowest free number per semester
title, description, domain	text	title NOT NULL	Project title, description and domain
leader, leader_roll	text	—	Leader display name and roll number
members	jsonb	DEFAULT '[]'	Member list (id, name, branch, roll, role)
memberCount, validationStatus	integer, text	DEFAULT 0, 'Pending'	Team size and validation state
mentor	text	—	Assigned mentor name(s)
problem_statement_id	text	FK → problem_statements	Reserved catalogue statement (ON DELETE SET NULL)

Table 3.4: Database Table – mentors

Field Name	Data Type	Key / Constraint	Description
id	text	Primary Key	Mentor code "M-###"
name	text	NOT NULL, Unique	Faculty name used as allocation key
email, phone	text	—	Contact details; phone is used for voice escalation
domains	text[]	—	Rank-ordered mentoring domains (index 0 = first choice)
skills, expertise	text[], text	—	Keyword signals for matching
max_teams, current_teams	integer	DEFAULT 3, 0	Capacity and current load
password_hash	text	—	bcrypt hash set at roster-gated sign-up
is_active	boolean	DEFAULT false	Coordinator approval flag

Table 3.5: Database Table – presentation_cycles and presentation_rubrics

Field Name	Data Type	Key / Constraint	Description
id	uuid	PK, gen_random_uuid ()	Cycle identifier
semester_id, title, type	text	title, type NOT NULL	Cohort, name and type (proposal, mid, final)
weightage	integer	NOT NULL DEFAULT 0	Marks the cycle contributes to the result
deadline	timestampt z	NOT NULL	Submission deadline
status	text	DEFAULT 'draft'	draft, active, closed or frozen
requires_synopsis, enable_mentor_eval	boolean	DEFAULT false	Mandatory upload and mentor component
rubrics.cycle_id	uuid	FK → cycles (CASCADE)	Rubric criteria of the cycle
rubrics.criterion, max_marks, weight	text, integer	NOT NULL	Criterion name, maximum marks and weight

Table 3.6: Database Table – evaluations

Field Name	Data Type	Key / Constraint	Description
id	bigint	Primary Key (identity)	Evaluation identifier
panel_id	varchar(64)	NOT NULL	Evaluator (panel or mentor) identifier
team_id, team_uid	bigint, uuid	FK → teams / teams_validation. uid	Evaluated team
scores_json, remarks	text	NOT NULL	Criterion scores q1…qN and mandatory remarks
receipt_id	varchar(64)	Unique	Receipt "REC-" + 10 hexadecimal characters
locked	boolean	DEFAULT true	Immutable after submission
semester_id	text	Unique (panel, team, semester)	One evaluation per panel, team and semester

Table 3.7: Database Table – results

Field Name	Data Type	Key / Constraint	Description
team	text	Primary Key	Team slug (one row per team)
team_uid	uuid	FK → teams_validation. uid	Canonical team link
title, domain, branch, mentor	text	—	Denormalised display data

Field Name	Data Type	Key / Constraint	Description
proposalMarks, midMarks, finalMarks	integer	—	Weightage-scaled stage marks
totalScore	integer	—	Sum of the stage marks
semester_id	text	—	Cohort scoping

Supporting tables include team_memberships (authoritative student-to-team link, role LEADER or MEMBER), team_join_requests, unassigned_students, proposals, panels and users, schedule_slots and slots, evaluation_drafts, presentation_submissions, presentation_evaluations with presentation_evaluation_scores, absent_students and makeup_slots, deadlines, governance_audit and project_override_logs, communication_history, communication_templates and email_deliveries, conversations and conversation_messages, github_data and github_agent_analysis, and the curated domains catalogue.

CHAPTER 4
IMPLEMENTATION AND USER INTERFACE

4.1 Technology Stack Used

The technology choices favour mature open-source components with strong typing at the API boundary, fast builds for five front-end applications, and managed infrastructure that a small team can operate.

Table 4.1: Technology Stack Summary

Layer	Technology	Version	Purpose
Frontend	React + React DOM	19.2	Component-based UI for all five applications
Frontend	Vite	8.x	Millisecond dev server and optimised production bundles
Frontend	Tailwind CSS	4.2	Utility-first styling with class-based dark mode
Frontend	React Router	7.18	Client-side routing and protected routes
Frontend	Framer Motion, Lenis, Lucide	12.x, 1.3, —	Animation, smooth scrolling and icon set shared by all portals
Frontend	Recharts, Chart.js	3.x, 4.5	Dashboards, analytics and repository charts
Frontend	SheetJS xlsx, PapaParse	0.20.3, 5.5	Excel and CSV import and export in the admin portal
Frontend	jsPDF + AutoTable	4.2, 5.0	Result reports, missed-slot reports and receipts as PDF
Frontend	Supabase JS, Axios	2.10x, 1.18	Student authentication, realtime pings, mentor API client
Backend	Python, FastAPI, Uvicorn	3.12, 0.115–0.116	Asynchronous REST services with OpenAPI documentation
Backend	Pydantic, pydantic-settings	2.x	Request validation and environment configuration
Backend	SQLAlchemy (async), asyncpg, Alembic	2.x	ORM data access in the student and mentor services
Backend	supabase-py	2.15–2.18	PostgREST data access in the admin and panel services
Backend	python-jose, passlib / bcrypt	≥ 3.4.1, 4.2	HS256 JWTs and password hashing
Backend	slowapi, fastapi-mail, smtplib	0.1.9, 1.6.4	Rate limiting and e-mail delivery
AI / Data	OpenAI GPT-4o-mini, GPT-4.1-mini	API	Classification, enrichment, blueprint suggestions and chat reports

Layer	Technology	Version	Purpose
AI / Data	NVIDIA NIM Llama 3.1 8B, nv-embedqa-e5-v5	API	Alternative LLM and embeddings for mentor matching
AI / Data	pandas, openpyxl, python-docx, pypdf	—	Roster and catalogue ingestion from Excel, Word and PDF
AI / Data	RapidFuzz, Jellyfish	—	Fuzzy and phonetic roster matching
Integration	Google Calendar API v3, Twilio Voice	v3, —	Meet links for online slots; escalation voice calls
Database	Supabase PostgreSQL, Auth, Storage, Realtime	—	Primary store with row-level security and RPC functions
DevOps	Docker Compose, nginx (alpine)	—	Nine containers; static hosting and reverse proxy
DevOps	Cloudflare Tunnel, AWS EC2, S3 + CloudFront	2024.10.1	TLS edge without inbound ports; hosting; landing site
DevOps	GitHub Actions	—	CI/CD: test, build-check and SSH deployment

4.2 Intelligence and Automation Architecture

The intelligence layer follows one rule: **AI advises, people decide**. Every agent produces a recommendation, a classification or a report that a coordinator, mentor or evaluator reviews, and every agent has a deterministic fallback, so the platform remains fully functional when no AI key is configured. Table 4.2 lists the agents and background services.

Table 4.2: Intelligence and Automation Layer – Agent Responsibilities

Agent / Service	Trigger	Technique	Output
Mentor Allocation Engine	Coordinator request	Rank-aware domain matching over an 11-cluster taxonomy, availability load balancing, hard filters	Top-5 mentors per team with score breakdown and reason
PS Ingestion & Enrichment	Catalogue upload (DOCX, PDF, XLSX, CSV)	Header heuristics, pypdf / python-docx extraction, GPT-4o-mini parsing and batch classification (20 per call)	Clean statements with type, difficulty and structured description
Domain Classifier & Blueprint	Catalogue browse or request	GPT-4o-mini in JSON mode over a 21-domain list; token cost computed per call	Domain labels; blueprint with objectives, tech stack, deliverables and tips
GitHub Analytics Agent	Mentor or admin trigger; 60-second sync loop	External Repository Analysis API plus GitHub commits; fuzzy author-to-member matching	Quality, security, originality, architecture and documentation scores; contribution share
Roster Validation Agent	Roster upload	Normalisation plus RapidFuzz (Jaro-Winkler, Damerau-Levenshtein) and optional metaphone	valid, duplicate, review and invalid buckets

Agent / Service	Trigger	Technique	Output
Communication & Notification	Coordinator send; workflow events	Segment resolution, escaped template rendering, chunked SMTP	E-mails, portal notifications, delivery log
Escalation Agent	Every 60 seconds	Slot start versus evaluation and draft status	E-mail at +5 and +10 minutes; Twilio call at +15 minutes
Deadline Scheduler	Every hour	IST deadline parsing, 3/1/0-day windows, 20-hour de-duplication	Portal reminders to students and mentors
PMSbot Report	/report chat command	Metrics computed in Python; LLM (JSON mode, temperature 0) extracts agreed tasks from ≤ 400 messages	Team status card posted in the conversation

4.2.1 Mentor Allocation Engine

Mentors pick their mentoring domains at sign-up from a searchable catalogue, and the order of selection becomes their preference rank. The engine resolves team and mentor labels — including combined labels such as "Machine Learning / AI" — into an 11-cluster taxonomy (AI and data science, web and software, IoT and embedded, cybersecurity, cloud and DevOps, blockchain, mobile, databases, AR/VR and games, business, healthcare). Mentors at full capacity or with no domain overlap are removed by hard filters; the rest are scored and returned with a human-readable reason such as "Rank #1 exact domain match: 'Computer Vision'; 1/3 teams loaded". The engine is deliberately deterministic and explainable: a coordinator overriding it can see exactly why each mentor was suggested.

Table 4.3: Mentor Allocation Scoring Weights

Signal	Ranked-Domain Model (current)	Rule Model (portal services)	Computation
Domain fit	80%	40%	Exact match at rank i: max(0.6, 1 − 0.1i); same cluster: max(0.3, 0.6 − 0.1i); no overlap: rejected
Keyword / skill overlap	—	30%	Share of required skills found in the mentor's skills, expertise and domains
Experience	—	10%	min(years ÷ 15, 1)
Availability	20%	20%	1 − current teams ÷ maximum teams
Embedding similarity	—	Optional	Hybrid score = 0.7 × rule + 0.3 × cosine similarity when an embedding key is present

4.2.2 Problem-Statement Ingestion and AI Enrichment

The coordinator uploads a catalogue in any common format. Spreadsheets are parsed with column heuristics (title, description, domain, difficulty, year, objectives, datasets, deliverables); Word and PDF files are converted to text and, when an OpenAI key is present, parsed by GPT-4o-mini into structured statements, with a rule-based parser as fallback. Titles are de-duplicated case-insensitively, batches of 20 statements are classified in parallel into hardware or software and easy, medium or hard, and short descriptions are enriched in the background into a structured brief. Model output is coerced onto the curated domain catalogue, so an unrecognised label is dropped rather than stored. On the student side, statements without a domain are classified on the fly, and the "AI suggestions" action returns a project blueprint together with the token cost of the call.

4.2.3 GitHub Repository Analytics Agent

When a team saves its repository URL, the agent queues an analysis job on the Repository Analysis API, polls its status (queued, running, completed or failed) and stores five scores — quality, security, originality, architecture and documentation — with the detected tech stack and the full report in github_agent_analysis. Git authors are matched to team members by e-mail, name or roll number, which yields each student's commit count and contribution share. Coordinators additionally get duplicate-commit, inactivity and repository-privacy reports per batch. All records are keyed by the canonical team_uid, and the manual analysis endpoint is rate-limited to ten calls per minute.

4.2.4 Communication, Reminder and Escalation Agents

The communication agent resolves ten audience segments (all students, valid or invalid teams, team leaders, unassigned students, incomplete teams, teams without mentors, mentors, panel members, a specific team) and nine lifecycle templates. Merge fields such as {team_name}, {project_title}, {leader_name} and {mentor_name} are HTML-escaped before substitution, mail is sent in chunks of 25 per SMTP connection, and every send writes communication_history plus one email_deliveries row per recipient. The deadline scheduler posts reminders three days, one day and on the day of each deadline. The escalation agent checks the day's slots every 60 seconds; if no evaluation or draft exists, it sends a first e-mail at +5 minutes, an urgent e-mail at +10 minutes and places an automated Twilio voice call to the evaluator at +15 minutes.

4.2.5 PMS-Chat and PMSbot

PMS-Chat provides role-aware conversations on three channels — mentors and students, panels and teams, and all portals — with standing threads per team, read receipts and slash commands. Message bodies are always fetched through the authenticated APIs; the Realtime table chat_events carries only a content-free ping. The /report command builds a status card whose numbers are computed in Python, while an LLM (GPT-4.1-mini or Llama 3.1 8B) only extracts agreed tasks from the conversation. The module is controlled by a build-time flag and per-portal switches in semester_config.

4.3 Core Algorithms and Integrity Mechanisms

- **Atomic problem-statement reservation:** the PostgreSQL function reserve_problem_statement_atomic locks the statement row and the team row with SELECT … FOR UPDATE, counts the other teams holding the statement (including legacy rows matched by normalised title), raises "problem statement is at capacity" when the count reaches two, and otherwise links the team and copies the title, description and domain — all inside one transaction, so two leaders racing for the last spot can never both succeed.
- **Team numbering:** a new team receives the lowest free team_number in its semester. A unique index turns a race between two students into a clean conflict, and the service retries with the next number up to five times.
- **Time-bound evaluation window:** submission is allowed only when slot_start − 2 h ≤ now ≤ slot_end. Each criterion must satisfy 0 ≤ score ≤ max_marks and remarks are mandatory; the normalised score is round(Σ scores ÷ Σ max_marks × 100).
- **Rubric precedence:** the rubric is taken from the cycle's presentation_rubrics, then the semester's evaluation_parameters, then the global evaluation_parameters table, and finally a four-criterion default of ten marks each.
- **Results aggregation:** for each cycle, the stage mark is round(mean of normalised scores ÷ 100 × cycle weightage), mapped to proposal, mid or final marks by cycle type; the total is their sum, and absentee records of the team are marked completed.
- **Freeze and locks:** middleware rejects writes with "System is frozen" when a batch is locked, rejects evaluation writes when marks are locked and rejects team changes when teams are locked.

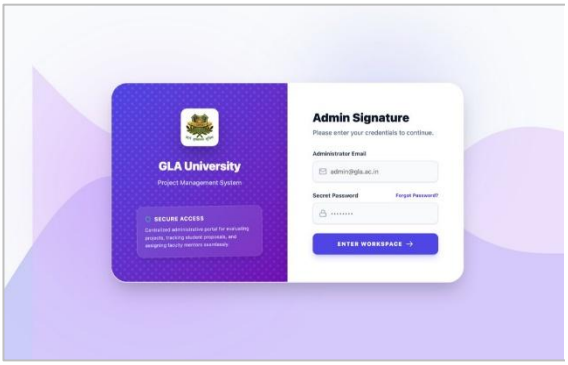
4.4 Frontend Implementation

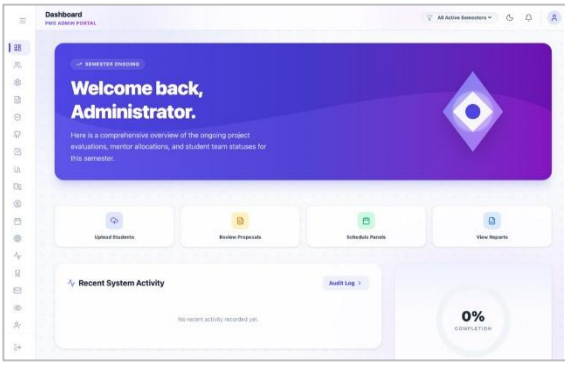
All five applications are React single-page apps that share a visual language — Tailwind CSS with light and dark themes, Framer Motion transitions, Lucide icons and first-login walkthrough tours. Each portal handles authentication in the way its backend expects: the student portal uses Supabase Auth with PKCE and a Google sign-in restricted to @gla.ac.in, the admin portal renews Supabase sessions through a single in-flight refresh, and the mentor portal's Axios client silently refreshes expired tokens and retries once. The panel portal lazy-loads every page and registers a service worker for evaluation notifications.

4.4.1 Admin (Coordinator) Portal

The admin portal is the control plane, with 24 routed screens. Key screens are:

- **Dashboard** with quick actions, recent system activity from the audit log and a completion ring.
- **Student and Mentor Master Data** with drag-and-drop Excel/CSV import, validation and export, and **Semester Settings** for batches, bounds, cycles and locks.
- **Proposals** with a document viewer and approve, reject or revise actions; **Team Validation** with size rules, overrides, renumbering and join requests.
- **Project Allocation** for catalogue import, AI suggestions and bulk allotment that spreads teams across statements before doubling up; **Mentor Allocation** with assignment e-mails.
- **Presentation Cycles, Schedule** (Meet links, slot sync) and **Evaluations** (live feed and alerts); **Results** with jsPDF reports and publishing.
- **Communication** and **Per-Team Mail**, **Governance** (deadlines, freeze, audit log, Excel master export), **Absent Students**, **Extra Activities**, **Repo Analysis** and **PMS-Chat**.



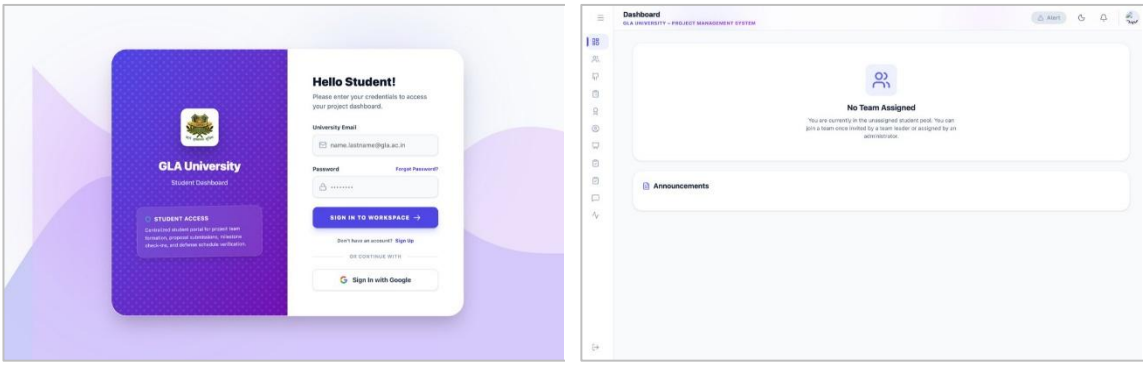


(a) Admin sign-in *(b) Coordinator dashboard*

Figure 4.1: Admin Portal – (a) Sign-in and (b) Dashboard

4.4.2 Student Portal

The student portal guides a student from sign-up to results. The dashboard shows the current stage, open deadlines and announcements, with banners when the system or team formation is locked. **Team & Proposal** handles team creation, join requests, member changes and proposal submission; **Browse & Reserve Topics** shows year-filtered statements with "1 spot left" and "full" badges; and **Milestones, Presentations, GitHub Analysis, Evaluations, Results, Feedback, Mentor, Notifications, Extra Activities** and **Profile** complete the journey.

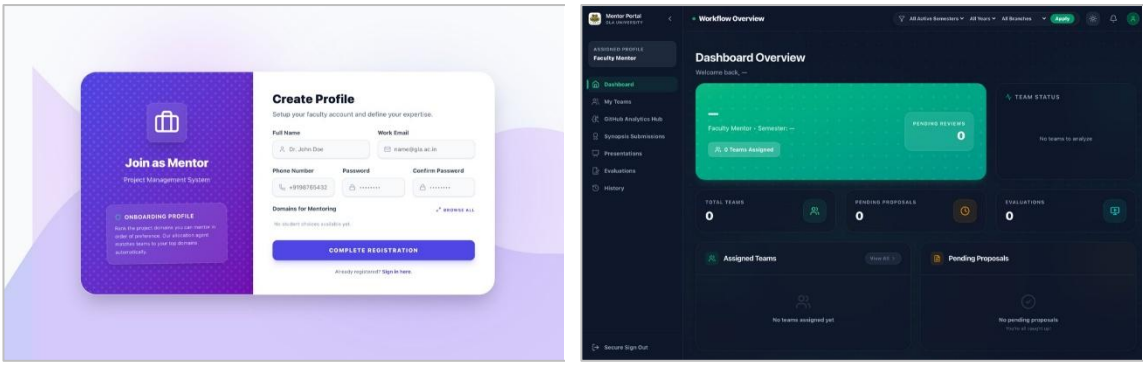


(a) Student sign-in *(b) Unassigned student dashboard*

Figure 4.2: Student Portal – (a) Sign-in with Google SSO and (b) Dashboard

4.4.3 Mentor Portal

Mentors create a faculty profile and choose ranked mentoring domains, then wait for coordinator approval. The **Dashboard** summarises assigned teams, pending proposals and evaluations; **My Teams** opens a detailed team view with the project blueprint; the **GitHub Analytics Hub** shows repository scores and per-student contribution; **Synopsis Submissions** provides a form builder (short and long answers, choices, GitHub link, file upload) to assign and verify milestones; and **Presentations, Evaluations** and **History** cover cycle evaluation within rubric bounds.



(a) Mentor sign-up with ranked domains *(b) Mentor dashboard*

Figure 4.3: Mentor Portal – (a) Profile Creation and (b) Dashboard

4.4.4 Panel Portal

The panel portal is the narrowest interface because it is used under time pressure. **My Slots** refreshes the day's queue every 30 seconds; **Parameters** shows the rubric built for each team's cycle; **Time-Bound Evaluation** runs a 30-minute countdown re-synchronised with the server every 15 seconds, auto-saves drafts and raises browser notifications whose actions extend the slot by 5 or 10 minutes; **Missed** handles reschedule requests; and the **Secure Submission Receipt** can be downloaded as a PDF with its receipt ID and SHA-256 signature.

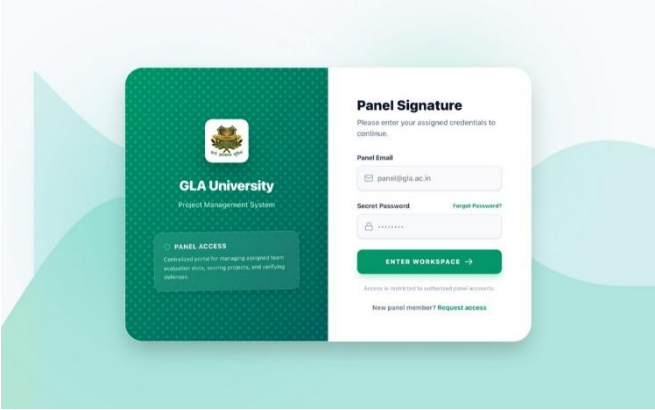


Figure 4.4: Panel Portal – Evaluator Sign-in

4.5 Deployment and DevOps

The platform runs as nine Docker Compose containers on one AWS EC2 host: a Cloudflare Tunnel connector, four nginx front-end containers and four Uvicorn back-end containers. All ports are bound to 127.0.0.1, so the only path from the internet is the outbound tunnel; Cloudflare routes admin, student, mentor and panel.glapms.in to the matching nginx container. The public landing site is built separately and served from Amazon S3 behind CloudFront.

Delivery is automated with GitHub Actions. The **Deploy EC2** workflow runs on every push and pull request to main: a test job runs pytest, ESLint for all five apps and the node:test suites; a build-check job builds every front-end and byte-compiles the backend; and the deploy job verifies that required secrets exist, connects to EC2 over SSH, writes per-service environment files with separate JWT secrets, rebuilds the containers and restarts them. A second workflow publishes the landing site to S3 and invalidates CloudFront. Database backups are taken daily by Supabase with seven-day retention, and a restore runbook is kept in the repository.

CHAPTER 5
SOFTWARE TESTING

5.1 Testing Strategy

Testing combines automated unit and contract tests, scenario-based black-box tests through the portals, white-box tests of the evaluation and allocation code paths, and security and concurrency testing of the database layer. Pure logic — allocation rules, chat timelines, reminder filtering, roster matching, template rendering and cohort-scoped team resolution — is kept in side-effect-free modules so that it can be unit-tested without a database. The CI pipeline runs the backend suites, the admin and student node:test suites, ESLint for all five applications and a production build of every front-end on each push and pull request; a failing step blocks deployment.

5.2 Automated Unit and Contract Tests

Table 5.1: Automated Test Suites

Suite	Location	Cases	What is Verified
Admin portal logic	apps/admin/tests (9 files)	79	Chat timeline and delivery states, commit attribution, chat channels and threads, PS capacity of two with spread-before-double allocation, team-number display, proposal serial numbers
Student portal logic	apps/student/tests (7 files)	17	Closed-deadline reminders stay silent, immutable team references for GitHub data, capacity badges, partial-fill indicator
Communication and roster	backend/tests (5 files)	42	Template rendering and registries, scheduled-send offsets, name and e-mail normalisation, fuzzy duplicate routing, recall and precision harness
Student API	student-api/tests (3 files)	8	Team-creation regressions, reservation capacity and RPC contract, background e-mail session ownership
Mentor API	mentor-api/tests (1 file)	16	Domain catalogue parsing: ampersands, misspellings, JSON lists, placeholders
Panel API	panel-api/tests (1 file)	7	Cohort-scoped team_uid resolution across concurrently active batches
Total	26 files	169	—

5.3 Black Box Test Cases

Table 5.2: Black Box Test Cases – Portal Workflows

TC ID	Test Description	Input	Expected Output	Result
1	Sign-up with a non-university e-mail	user@gmail.com	Rejected with INVALID_DOMAIN	Pass
2	Sign-up by a student not on the roster	@gla.ac.in address not uploaded	Rejected with STUDENT_NOT_PRE_REGISTERED	Pass
3	Google sign-in by an unregistered account	OAuth session of a non-roster user	Signed out with a "not pre-registered" message	Pass
4	Create a team while already in a team	Student with an active membership	409 "You are already in a team"	Pass
5	Reserve a statement held by two teams	Third team selects a full statement	409 "problem statement is at capacity"	Pass
6	Proposal submitted by a non-leader	Member account	403 "Only the team leader can submit proposals"	Pass
7	First proposal after the deadline	Submission after the due date	400 "Proposal submission deadline has passed"	Pass
8	Resubmission after rejection	Latest proposal is Rejected	Accepted; deadline check skipped	Pass
9	Panel submission outside the slot window	Submit after the slot has ended	403 time-bound control message	Pass
10	Score above the criterion maximum	Criterion maximum 10, score 12	422 "Score must be 0–10"	Pass
11	Final submission without remarks	Blank remarks	422 "Remarks required"	Pass
12	Any write while the batch is frozen	Data API write with is_locked = true	403 "System is frozen. No modifications allowed."	Pass

5.4 White Box Test Cases

Table 5.3: White Box Test Cases – Evaluation and Allocation Logic

TC ID	Test Description	Condition Tested	Expected Behaviour	Result
1	Rubric precedence	No cycle rubric; semester parameters exist	Semester parameters copied into the cycle rubric; keys q1…qN	Pass
2	Rubric fallback	No cycle, semester or global parameters	Default of four criteria × 10 marks (maximum 40)	Pass
3	Concurrent team creation	Two students claim the same number	Unique-index conflict; retry with the next number (≤ 5 times)	Pass

TC ID	Test Description	Condition Tested	Expected Behaviour	Result
4	Reservation race	Two leaders take the last spot together	Row lock serialises; second call raises the capacity error	Pass
5	Results aggregation	Evaluations in proposal, mid and final cycles	Stage mark = round(avg ÷ 100 × weightage); total = sum	Pass
6	Draft after submission	Draft saved for an evaluated team	400 “Already submitted; draft locked”	Pass
7	Slot extension	Extend a slot by 10 minutes	Later slots on the same panel or room shift; online rooms ignored	Pass
8	Escalation suppression	Draft exists at +12 minutes	No e-mail or call; each stage fires once per slot	Pass
9	Mentor score bounds	Total above the cycle maximum	400 score out of range	Pass
10	Admin role trust	role = admin only in user_metadata	403 “Administrator access required”	Pass

5.5 Security and Concurrency Testing

A structured, incident-driven security review of the database, authentication, API and secrets layers was carried out in September 2026 using a purpose-built auditor (scripts/security_audit) that inspects row-level security, policies, grants, SECURITY DEFINER functions, storage buckets and authentication settings, and probes the API as an anonymous client. The review drove six database migrations and a set of code fixes; Table 5.4 summarises the verification checks after remediation.

Table 5.4: Security Testing Results

Check	Method	Result after Remediation
Anonymous table access	GET /rest/v1/students with the publishable key	401 permission denied
Storage exposure	Public object URL and bucket listing	Bucket not found; empty list
Permissive policies	Static audit of 71 public tables	Only the content-free chat_events read policy remains
Role escalation at sign-up	Sign-up carrying role = admin in user metadata	Ignored; role read only from app_metadata
Unauthenticated write routes	Route-by-route scan of all four services	None, apart from auth bootstrap routes
SQL injection	Review of every raw SQL statement	Values bound as parameters
Secrets in client bundles	Review of VITE_ variables and built assets	Admin key moved behind a server-side proxy

Check	Method	Result after Remediation
Token reuse across services	Decode a token on another service	Rejected by audience and per-service secret

Concurrency and integrity were verified with the row-locking reservation function, the unique team-number index with retries, and verification queries shipped with the uid_canonical migrations, which must return zero rows — for example, no evaluation, result or slot may reference a team from a different cohort than its presentation cycle.

CHAPTER 6
CONCLUSION

6.1 Current Status and Future Work

GLA PMS demonstrates that an academic project lifecycle with hundreds of students, several concurrent cohorts and many stakeholders can be run on one coherent platform. The project set out to remove fragmented records, topic contention, manual matching, evaluation overhead and missing accountability, and each of these is now handled by a dedicated, tested mechanism. The four-portal, four-service architecture keeps each user population inside its own trust boundary, while the shared, semester-scoped schema keeps every team, mark and message consistent.

The system ensures:

- One authoritative roster and team registry per semester, with roll-number keys and canonical team identifiers.
- Conflict-free topic allocation, with at most two teams per problem statement enforced by the database under concurrency.
- Explainable, capacity-aware mentor recommendations that the coordinator can accept, override or lock.
- Time-bound, tamper-evident evaluations with drafts, locks and receipts, and automatic weightage-scaled results.
- Logged, segment-based communication with automatic reminders and escalation of late evaluations.
- Defence-in-depth security verified by audit: deny-by-default row-level security, audience-scoped tokens and server-side secrets.

Future work includes moving the mentor and panel services fully onto the canonical team registry and retiring the legacy tables, enabling PMS-Chat in production, extending automated and end-to-end tests to every service, adding a shared cache and uniform rate limiting, plagiarism checks for synopsis documents, an analytics view that compares cohorts across semesters, and a mobile-friendly evaluator mode for in-hall presentations.

CHAPTER 7
SUMMARY

7.1 Summary

The GLA PMS project delivers an AI-assisted, multi-portal platform for the complete academic project lifecycle of the Department of Computer Engineering & Applications. Coordinators, students, mentors and panel evaluators each work in a dedicated React portal backed by an independent FastAPI service, and all services share one Supabase PostgreSQL schema in which every cohort is isolated by semester.

The platform automates the stages that previously consumed the most manual effort. Rosters are imported with fuzzy duplicate detection; problem-statement catalogues are ingested from Word, PDF or Excel and classified with GPT-4o-mini; teams reserve topics through an atomic database function; mentors are recommended by an explainable rank-aware engine; presentations are scheduled with Google Meet links; panels evaluate inside a server-enforced window that issues receipts; and results are computed from locked evaluations and published with a single action. Communication, reminders, escalations and repository analytics run as agents that advise people rather than replace them.

The system is containerised, deployed behind a Cloudflare Tunnel on AWS EC2 through a GitHub Actions pipeline, and protected by layered security that was verified by a structured audit. With 169 automated test cases and a production deployment serving two concurrent cohorts, GLA PMS provides a strong foundation for future enhancements such as in-production messaging, cross-semester analytics and deeper AI-assisted review.

REFERENCES

1. M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015.
2. N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” in Proc. USENIX Annual Technical Conference (FREENIX Track), 1999.
3. W. E. Winkler, “String Comparator Metrics and Enhanced Decision Rules in the Fellegi-Sunter Model of Record Linkage,” in Proc. Section on Survey Research Methods, American Statistical Association, 1990, pp. 354–359.
4. F. J. Damerau, “A Technique for Computer Detection and Correction of Spelling Errors,” Communications of the ACM, vol. 7, no. 3, pp. 171–176, 1964.
5. V. I. Levenshtein, “Binary Codes Capable of Correcting Deletions, Insertions, and Reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.
6. I. P. Fellegi and A. B. Sunter, “A Theory for Record Linkage,” Journal of the American Statistical Association, vol. 64, no. 328, pp. 1183–1210, 1969.
7. S. Rose, D. Engel, N. Cramer and W. Cowley, “Automatic Keyword Extraction from Individual Documents,” in Text Mining: Applications and Theory, Wiley, 2010, pp. 1–20.
8. OpenAI, “GPT-4o System Card,” arXiv:2410.21276, 2024.
9. Llama Team, Meta AI, “The Llama 3 Herd of Models,” arXiv:2407.21783, 2024.
10. S. Newman, Building Microservices: Designing Fine-Grained Systems, 2nd ed., O’Reilly Media, 2021.
11. OWASP Foundation, “OWASP Top 10:2021,” 2021. Available: https://owasp.org/Top10/
12. PostgreSQL Global Development Group, “Explicit Locking,” PostgreSQL Documentation. Available: https://www.postgresql.org/docs/current/explicit-locking.html
13. Supabase, “Row Level Security” and “Auth” documentation. Available: https://supabase.com/docs
14. FastAPI Documentation. Available: https://fastapi.tiangolo.com
15. React Documentation (https://react.dev) and Vite Documentation (https://vitejs.dev).
16. Cloudflare, “Cloudflare Tunnel” documentation. Available: https://developers.cloudflare.com/cloudflare-one/
17. GitHub, “GitHub Actions” documentation. Available: https://docs.github.com/actions
18. Google, “Google Calendar API – Events and conferenceData,” and Twilio, “TwiML Voice: Say.” Available: https://developers.google.com/calendar/api and https://www.twilio.com/docs/voice/twiml

APPENDIX A: LIST OF ABBREVIATIONS

Abbreviation	Full Form	Abbreviation	Full Form
AI	Artificial Intelligence	NIM	NVIDIA Inference Microservices
AIML	Artificial Intelligence and Machine Learning	OAuth	Open Authorization
API	Application Programming Interface	PDF	Portable Document Format
CEA	Computer Engineering and Applications	PKCE	Proof Key for Code Exchange
CI/CD	Continuous Integration / Continuous Deployment	PMS	Project Management System
CORS	Cross-Origin Resource Sharing	PS	Problem Statement
CSV	Comma-Separated Values	REST	Representational State Transfer
DFD	Data Flow Diagram	RLS	Row-Level Security
DOCX	Word Open XML Document	RPC	Remote Procedure Call
EC2	Elastic Compute Cloud	S3	Simple Storage Service
ER	Entity Relationship	SMTP	Simple Mail Transfer Protocol
GLAU	GLA University	SPA	Single-Page Application
HW / SW	Hardware / Software	SQL	Structured Query Language
IST	Indian Standard Time	SSO	Single Sign-On
JSON	JavaScript Object Notation	TLS	Transport Layer Security
JWT	JSON Web Token	UID	Unique Identifier
LLM	Large Language Model	UML	Unified Modeling Language
MCQ	Multiple-Choice Question	UI / UX	User Interface / User Experience

APPENDIX B: SAMPLE API ENDPOINTS

The four services expose about 325 route declarations (admin-api 56, panel-api 49, student-api 117, mentor-api 105). Every endpoint except the authentication bootstrap routes requires a Bearer token; the admin service applies its guard at application level so that a new route is protected by default. Representative endpoints are listed below.

Method	Endpoint	Service	Description
POST	/api/auth/login	admin	Coordinator sign-in through Supabase Auth
GET	/api/data/{table}	admin	Semester-scoped read over 39 whitelisted tables
POST	/api/data/{table}/upsert	admin	Idempotent bulk upsert with freeze check
POST	/api/admin/projects/import-ps	admin	Catalogue import with AI classification
POST	/api/admin/communication/send-per-team	admin	Per-team mail merge with attachments
POST	/api/events/ensure-slot-meet	admin	Create, move or clear Google Meet events
POST	/api/auth/signup	student	Roster-gated sign-up with e-mail confirmation
GET	/api/student/problem-statements	student	Year-filtered catalogue with capacity flags
POST	/api/student/team/create	student	Create a team with the lowest free number
POST	/api/student/team/reserve-ps	student	Atomic reservation (capacity 2)
POST	/api/student/proposals	student	Leader-only proposal with deadline check
GET	/api/mentor/teams/{team_id}/github	mentor	Repository analytics for an assigned team
POST	/api/mentor/presentation-cycles/{id}/evaluate	mentor	Cycle evaluation within rubric bounds
GET	/evaluation/session/{team_id}	panel	Session with rubric and remaining time
PUT	/evaluation/draft/{team_id}	panel	Auto-save draft scores inside the window
POST	/evaluation/panel	panel	Locked final submission returning a receipt
POST	/evaluation/extend/{team_id}	panel	Extend a slot and shift later slots